

Reviewer Pack — Schur-Weyl Decomposition Complexity of Symmetric Powers of P...

Entry 8a09bc5f4a1c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 00:40:22 UTC

Schur-Weyl Decomposition Complexity of Symmetric Powers of Permanent vs Determinant

Entry ID: 8a09bc5f4a1c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 00:40:22 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Symmetric Power Decomposition of Homogeneous Polynomials

Statement:

For all $n \geq 2$ and $m < n^{\{1.5\}}$, the number of irreducible components in the decomposition of $\text{Sym}^k(\text{perm}_n)$ under $\text{GL}(n)$ exceeds that of $\text{Sym}^k(\text{det}_m)$ by a factor of $\Omega(n^{\{k-1\}})$ for $k = \lceil \log n \rceil$.

Rationale (proposer's reasoning):

Schur-Weyl duality decomposes symmetric powers into irreducibles, revealing structural asymmetry between permanent and determinant. The conjecture posits that permanent's decomposition exhibits exponentially more components than determinant's under symmetric power operations, reflecting inherent algebraic complexity.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 60a67d6e126354b6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def factorial(n):
        if n == 0 or n == 1:
            return 1
        result = 1
        for i in range(2, n + 1):
            result *= i
        return result

    def hook_length_formula(n, k):
        hook_lengths = [[0] * (n - i) for i in range(n)]
        for i in range(n):
            for j in range(n - i):
                hook_lengths[i][j] = n - i + 1 + j
        det = 1
        for i in range(n):
            for j in range(n - i):
                det *= factorial(hook_lengths[i][j])
        return det

    def count_irreducible_components(n, k):
        perm_n_decomp = hook_length_formula(n, k)
        det_m_decomp = hook_length_formula(k, k)
        return perm_n_decomp, det_m_decomp

    n = random.randint(2, 40)
    m = random.randint(1, int(math.sqrt(n)) - 1)
    k = math.ceil(math.log(n))

    perm_n_decomp, det_m_decomp = count_irreducible_components(n, k)

    conjecture_holds = perm_n_decomp > det_m_decomp * n**(k-1)
    counterexample = "" if conjecture_holds else f"perm_n_decomp={perm_n_decomp}, det_m_decomp={det_m_decomp}"
```

```

    return {
        "metric_name": "irreducible_components_ratio",
        "metric_value": perm_n_decomp / det_m_decomp,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [random.getrandbits(32) for _ in range(10)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    total_metric_value = sum(r["metric_value"] for r in results)
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={total_metric_value/len(results)} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={total_metric_value/len(results)} std=0.0 support_fraction={support_fraction}")
    else:
        for r in results:
            if not r["conjecture_holds"]:
                counterexample = r["counterexample"]
                first_failing_seed = seed
                break
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_91dc2157.py", line 68, in <module>
    result = run_trial(seed)
             ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_91dc2157.py", line 56, in run_trial
    "metric_value": perm_n_decomp / det_m_decomp,
                     ~~~~~^~~~~~
OverflowError: integer division result too large for a float

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with overflow error, preventing metric evaluation. Support condition cannot be verified without valid results. | next: Implement arbitrary-precision arithmetic or logarithmic transformation for the metric calculation to handle large values

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	69201
2	preregistration	ollama_remote	qwen3:8b	0	0	33685
3	novelty	ollama_remote	qwen3:8b	0	0	27569
4	novelty	ollama_remote	qwen3:8b	0	0	28921
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14144
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6534
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11008
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8067
9	judge	ollama_remote	qwen3:8b	0	0	27578

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 226707 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/8a09bc5f4a1c.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/8a09bc5f4a1c.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/8a09bc5f4a1c.tar.gz (if generated)